

AI6102: Machine Learning Methodologies & Applications

ZHANG MINGHAO
MINGHAO008@e.ntu.edu.sg**Question 1 (10 marks):**

Consider a multi-class classification problem of C classes. Based on the parametric forms of the conditional probabilities of each class introduced on the 39th Page (“Extension to Multiple Classes”) of the lecture notes of L4, derive the learning procedure of **regularized logistic regression** for multi-class classification problems.

Hint: define a loss function by borrowing an idea from binary classification, and derive the gradient descent rules to update $\{\mathbf{w}^{(c)}\}$ ’s.

Answer:**1. Model and notation**

Let the training set be

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N, \quad \mathbf{x}_i \in \mathbb{R}^d, \quad y_i \in \{0, 1, \dots, C-1\}.$$

Following slide 39 of L4, class 0 is the reference class. We fix $\mathbf{w}^{(0)} = \mathbf{0}$ and learn

$$W = \{\mathbf{w}^{(1)}, \dots, \mathbf{w}^{(C-1)}\}.$$

If an intercept is included, a constant feature can be appended to $\mathbf{x}_i$. Here, the stated L_2 penalty applies to all coordinates of the learned parameter vectors.

Define

$$Z_i = 1 + \sum_{k=1}^{C-1} \exp\left(-(\mathbf{w}^{(k)})^\top \mathbf{x}_i\right).$$

The conditional class probabilities are

$$p_{i0} := P(y_i = 0 \mid \mathbf{x}_i; W) = \frac{1}{Z_i}, \tag{1}$$

$$p_{ic} := P(y_i = c \mid \mathbf{x}_i; W) = \frac{\exp(-(\mathbf{w}^{(c)})^\top \mathbf{x}_i)}{Z_i}, \quad c = 1, \dots, C-1. \tag{2}$$

These probabilities satisfy $\sum_{c=0}^{C-1} p_{ic} = 1$.

Introduce the indicator

$$t_{ic} = \mathbf{1}\{y_i = c\}, \quad \sum_{c=0}^{C-1} t_{ic} = 1.$$

2. Regularized loss function

As in binary logistic regression, we minimize the negative conditional log-likelihood. Assuming independent training examples, the conditional likelihood is

$$L(W) = \prod_{i=1}^N \prod_{c=0}^{C-1} p_{ic}^{t_{ic}}.$$

Therefore, the multiclass cross-entropy loss is

$$-\log L(W) = -\sum_{i=1}^N \sum_{c=0}^{C-1} t_{ic} \log p_{ic}.$$

Adding an L_2 regularization term gives the optimization problem

$$\min_W E(W) = -\sum_{i=1}^N \sum_{c=0}^{C-1} t_{ic} \log p_{ic} + \frac{\lambda}{2} \sum_{c=1}^{C-1} \|\mathbf{w}^{(c)}\|_2^2, \quad \lambda > 0.$$

The data loss is summed over the training examples, consistently with the convention in L4.

Since

$$\log p_{i0} = -\log Z_i$$

and, for $c > 0$,

$$\log p_{ic} = -(\mathbf{w}^{(c)})^\top \mathbf{x}_i - \log Z_i,$$

the loss for example i can be rewritten as

$$\ell_i(W) = -\sum_{c=0}^{C-1} t_{ic} \log p_{ic} \tag{3}$$

$$= \sum_{c=1}^{C-1} t_{ic} (\mathbf{w}^{(c)})^\top \mathbf{x}_i + \left(\sum_{c=0}^{C-1} t_{ic} \right) \log Z_i \tag{4}$$

$$= \sum_{c=1}^{C-1} t_{ic} (\mathbf{w}^{(c)})^\top \mathbf{x}_i + \log Z_i. \tag{5}$$

Hence,

$$E(W) = \sum_{i=1}^N \left[\sum_{c=1}^{C-1} t_{ic} (\mathbf{w}^{(c)})^\top \mathbf{x}_i + \log Z_i \right] + \frac{\lambda}{2} \sum_{c=1}^{C-1} \|\mathbf{w}^{(c)}\|_2^2.$$

3. Gradient derivation

Fix a class $r \in \{1, \dots, C-1\}$. The derivative of the linear term in $\ell_i(W)$ is

$$\nabla_{\mathbf{w}^{(r)}} \left[\sum_{c=1}^{C-1} t_{ic} (\mathbf{w}^{(c)})^\top \mathbf{x}_i \right] = t_{ir} \mathbf{x}_i.$$

For the normalization term, the chain rule gives

$$\nabla_{\mathbf{w}^{(r)}} \log Z_i = \frac{1}{Z_i} \nabla_{\mathbf{w}^{(r)}} Z_i \tag{6}$$

$$= \frac{-\exp(-(\mathbf{w}^{(r)})^\top \mathbf{x}_i) \mathbf{x}_i}{Z_i} \tag{7}$$

$$= -p_{ir} \mathbf{x}_i. \tag{8}$$

Thus,

$$\nabla_{\mathbf{w}^{(r)}} \ell_i(W) = (t_{ir} - p_{ir}) \mathbf{x}_i.$$

The regularization term contributes

$$\nabla_{\mathbf{w}^{(r)}} \left[\frac{\lambda}{2} \sum_{c=1}^{C-1} \|\mathbf{w}^{(c)}\|_2^2 \right] = \lambda \mathbf{w}^{(r)}.$$

Combining the two parts, we obtain

$$\nabla_{\mathbf{w}^{(r)}} E(W) = \sum_{i=1}^N (t_{ir} - p_{ir}) \mathbf{x}_i + \lambda \mathbf{w}^{(r)}, \quad r = 1, \dots, C-1.$$

4. Gradient descent and learning procedure

Let $\eta_t > 0$ be the learning rate at iteration t , and let $p_{ir}^{(t)}$ denote the probability computed using W_t . Gradient descent updates each learned class parameter as

$$\mathbf{w}_{t+1}^{(r)} = \mathbf{w}_t^{(r)} - \eta_t \left[\sum_{i=1}^N (t_{ir} - p_{ir}^{(t)}) \mathbf{x}_i + \lambda \mathbf{w}_t^{(r)} \right], \quad r = 1, \dots, C-1.$$

Equivalently,

$$\mathbf{w}_{t+1}^{(r)} = (1 - \eta_t \lambda) \mathbf{w}_t^{(r)} + \eta_t \sum_{i=1}^N (p_{ir}^{(t)} - t_{ir}) \mathbf{x}_i.$$

The sign follows from the negative exponent in the parameterization given on slide 39.

The complete batch gradient descent procedure is:

1. Choose $\lambda > 0$, a convergence tolerance $\varepsilon > 0$, and a maximum number of iterations. Initialize $\mathbf{w}_0^{(r)} = \mathbf{0}$ for $r = 1, \dots, C-1$, keeping $\mathbf{w}^{(0)} = \mathbf{0}$ fixed.
2. At iteration t , compute $Z_i^{(t)}$ and $p_{ic}^{(t)}$ for all training examples and classes.
3. Compute $\mathbf{g}_t^{(r)} = \nabla_{\mathbf{w}^{(r)}} E(W_t)$ for every $r = 1, \dots, C-1$.
4. Select a suitable learning rate η_t , for example using backtracking line search, and update all $\mathbf{w}^{(r)}$ simultaneously using the rule above.
5. Repeat until

$$\left(\sum_{r=1}^{C-1} \|\mathbf{g}_t^{(r)}\|_2^2 \right)^{1/2} \leq \varepsilon$$

or the maximum number of iterations is reached.

The objective is convex because each data-loss term is a log-sum-exp of affine functions plus a linear term. With $\lambda > 0$ and all learned coordinates regularized, the objective is strongly convex and has a unique minimizer. A suitable step-size rule allows gradient descent to converge to this minimizer.

For a new example $\mathbf{x}_*$, the predicted class is

$$\hat{y}_* = \arg \max_{c \in \{0, \dots, C-1\}} P(y = c \mid \mathbf{x}_*; \widehat{W}).$$

Question 2 (5 marks):

This is a hands-on exercise to use the SVC API of scikit-learn¹ to train a SVM with the linear kernel and the rbf kernel, respectively, on a binary classification dataset. The details of instructions are described as follows.

1. Download the a9a dataset from the LIBSVM Dataset page.

This is a preprocessed dataset of the Adult dataset in the UCI Irvine Machine Learning Repository², which consists of a training set (available here) and a test set (available here).

Each file (the train set or the test set) is a text format in which each line represents a labeled data instance as follows:

label index1:value1 index2:value2 ...

where “label” denotes the class label of each instance, “indexT” denotes the T-th feature, and valueT denotes the value of the T-th feature of the instance.

This is a sparse format, where only non-zero feature values are stored for each instance. For example, suppose given a data set, where each data instance has 5 dimensions (features). If a data instance whose label is “+1” and the input data instance vector is $[2 \ 0 \ 2.5 \ 4.3 \ 0]$, then it is presented in a line as

+1 1:2 3:2.5 4:4.3

Hint: scikit-learn provides an API (“`sklearn.datasets.load_svmlight_file`”) to load such a sparse data format. Detailed information is available here.

2. Regarding the linear kernel, show 3-fold cross-validation results in terms of classification accuracy on the training set with different values of the parameter C in $\{0.01, 0.05, 0.1, 0.5, 1\}$, respectively, in the following table. Note that for all the other parameters, you can simply use the default values or specify the specific values you used in your submitted PDF file.

Table 1: The 3-fold cross-validation results of varying values of C in SVC with linear kernel on the a9a training set (in accuracy).

$C = 0.01$	$C = 0.05$	$C = 0.1$	$C = 0.5$	$C = 1$
?	?	?	?	?

3. Regarding the rbf kernel, show 3-fold cross-validation results in terms of classification accuracy on the training set with different values of the parameter γ (i.e., σ^2 on the lecture notes) in $\{0.01, 0.05, 0.1, 0.5, 1\}$ and different values of the parameter C in $\{0.01, 0.05, 0.1, 0.5, 1\}$, respectively, in the following table. Note that for all the other parameters, you can simply use the default values or specify the specific values you used in your submitted PDF file.

Table 2: The 3-fold cross-validation results of varying values of γ and C in SVC with rbf kernel on the a9a training set (in accuracy).

¹Read Pages 63-64 of the lecture notes of L5 for reference

²The details of the original Adult dataset can be found here.

	$g = 0.01$	$g = 0.05$	$g = 0.1$	$g = 0.5$	$g = 1$
$C = 0.01$	?	?	?	?	?
$C = 0.05$	?	?	?	?	?
$C = 0.1$	?	?	?	?	?
$C = 0.5$	?	?	?	?	?
$C = 1$	?	?	?	?	?

Hint: there are no specific APIs that integrates cross-validation into SVMs in scikit-learn. However, you can use some APIs under the category “Model Selection → Model validation” to implement it. Some examples can be found here.

- Based on the results shown in Tables 1-2, determine the best kernel and the best parameter setting. Use the best kernel with the best parameter setting to train a SVM using the whole training set and make predictions on test set to generate the following table:

Table 3: Test results of SVC on the a9a test set (in accuracy).

	Specify which kernel with what parameter setting
Accuracy of SVMs	?

Answer:

1. Dataset and experimental protocol

The official LIBSVM a9a training set and a9a.t test set were used.³

The training set contains 32,561 examples and the test set contains 16,281 examples, with 123 features and labels in $\{-1, +1\}$.

Both files were loaded using `load_svmlight_file` with `n_features=123`, ensuring consistent feature dimensions. The supplied binary feature values were used without additional scaling, feature selection, or resampling. Sparse inputs were converted to dense arrays without changing their values.

All experiments used `sklearn.svm.SVC` in scikit-learn 1.7.2 (Python 3.10.9, NumPy 2.2.6, SciPy 1.15.3). The training set was divided using `StratifiedKFold` with `n_splits=3`, `shuffle=True`, and `random_state=42`. The same folds were used for all 30 candidate configurations. The validation fold sizes were 10,854, 10,854, and 10,853. For each fold, a new SVC was fitted on the other two folds and evaluated on the held-out fold.

For validation fold V_k , define

$$A_k = \frac{1}{|V_k|} \sum_{i \in V_k} \mathbf{1}\{\hat{y}_i^{(k)} = y_i\}, \quad \bar{A}_{CV} = \frac{1}{3} \sum_{k=1}^3 A_k.$$

Entries in Tables 1 and 2 are $100\bar{A}_{CV}$, expressed as percentages to four decimal places. Model selection used the unrounded mean accuracies.

Apart from the kernel and the grid parameters, SVC defaults were retained, except that `cache_size=512` MB was used. In particular, `tol=0.001`, `shrinking=True`, `probability=False`, `class_weight=None`, and `max_iter=-1`. All 90 cross-validation fits completed successfully without convergence warnings. The test set was reserved for final evaluation and was not used to select the kernel or hyperparameters.

³LIBSVM dataset source: <https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/binary.html>.

2. Linear kernel: Table 1

For the linear kernel

$$K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z},$$

`SVC(kernel='linear')` was evaluated at

$$C \in \{0.01, 0.05, 0.1, 0.5, 1\}.$$

Table 1: Three-fold cross-validation accuracy (%) of linear SVC on the a9a training set.

$C = 0.01$	$C = 0.05$	$C = 0.1$	$C = 0.5$	$C = 1$
84.3770	84.6688	84.7548	84.7394	84.7456

The highest mean accuracy for the linear kernel was 84.7548%, obtained with $C = 0.1$.

3. RBF kernel: Table 2

For the RBF kernel, the scikit-learn parameterization was used:

$$K(\mathbf{x}, \mathbf{z}) = \exp(-\gamma \|\mathbf{x} - \mathbf{z}\|_2^2).$$

All 25 combinations of

$$C, \gamma \in \{0.01, 0.05, 0.1, 0.5, 1\}$$

were evaluated.

Here, γ denotes the SVC argument `gamma`. For the lecture's RBF expression $\exp(-\|\mathbf{x} - \mathbf{z}\|^2 / (2\sigma^2))$, the corresponding relation is

$$\gamma = \frac{1}{2\sigma^2}.$$

The values specified in the assignment were passed directly to `gamma`.⁴

Table 2: Three-fold cross-validation accuracy (%) of RBF SVC on the a9a training set.

C	$\gamma = 0.01$	$\gamma = 0.05$	$\gamma = 0.1$	$\gamma = 0.5$	$\gamma = 1$
0.01	75.9190	81.9784	81.9877	75.9190	75.9190
0.05	83.1209	83.5816	83.4495	78.9841	75.9190
0.1	83.7351	83.9348	83.9133	80.5933	76.1832
0.5	84.3432	84.4968	84.6381	83.1946	78.9626
1	84.4599	84.7363	84.7271	83.5939	79.9730

The best RBF configuration was $C = 1$ and $\gamma = 0.05$, with mean cross-validation accuracy 84.7363%. Its three validation accuracies were 84.6785%, 84.8996%, and 84.6310%, respectively.

⁴SVC kernel definitions: <https://scikit-learn.org/1.7/modules/svm.html>.

4. Model selection and final test result: Table 3

Comparing all configurations in Tables 1 and 2, the selected model was

$$\boxed{\text{Linear kernel, } C = 0.1}$$

Its mean validation accuracy exceeded the best RBF-kernel result by 0.0184 percentage points. This identifies the best configuration within the specified grid under the stated cross-validation protocol.

A new SVC with this configuration was trained on all 32,561 training examples, keeping the other settings unchanged. It was then evaluated once on the 16,281 test examples, correctly classifying 13,841:

$$\text{Test accuracy} = \frac{13841}{16281} \times 100\% = 85.0132\%.$$

Table 3: Accuracy of the selected SVC on the a9a test set.

Selected configuration	Test accuracy (%)
Linear kernel, $C = 0.1$	85.0132

5. Experimental evidence

Figures 1 and 2 show terminal summaries of the saved experimental records. The cross-validation means were checked against the per-fold results, and the final test accuracy was verified against the saved predictions and test labels.

```

• (base) zhang@ZhangdeMacBook-Air question2 % ../.venv-q2/bin/python show_evidence.py
Q2: VERIFIED SAVED EXPERIMENT RESULTS (no retraining)
Source: /Users/zhang/Documents/ML/question2/results
Python 3.10.9 | scikit-learn 1.7.2
Training: 32561 x 123; no additional preprocessing
StratifiedKFold: 3 folds, shuffle=True, random_state=42
Checks passed: 30 configurations; 90 fits; no convergence warnings

LINEAR KERNEL: mean validation accuracy (%)
  C      0.01    0.05    0.1    0.5    1
Accuracy 84.3770 84.6688 84.7548 84.7394 84.7456

RBF KERNEL: mean validation accuracy (%)
  C / gamma  0.01    0.05    0.1    0.5    1
0.01 75.9190 81.9784 81.9877 75.9190 75.9190
0.05 83.1209 83.5816 83.4495 78.9841 75.9190
0.1 83.7351 83.9348 83.9133 80.5933 76.1832
0.5 84.3432 84.4968 84.6381 83.1946 78.9626
1 84.4599 84.7363 84.7271 83.5939 79.9730

Selected by unrounded CV accuracy: {'kernel': 'linear', 'C': 0.1}
Best mean CV accuracy: 84.7548%

```

Figure 1: Verified saved results for all 30 parameter configurations under the fixed three-fold cross-validation protocol.

```

• (base) zhang@ZhangdeMacBook-Air question2 % ../venv-q2/bin/python show_evidence.py test
Q2: VERIFIED SAVED EXPERIMENT RESULTS (no retraining)
Source: /Users/zhang/Documents/ML/question2/results
Original execution log: experiment.log
COMPLETE 30 /30 {'kernel': 'rbf', 'C': 1.0, 'gamma': 0.5} 0.8359387635015699
SELECTED {'kernel': 'linear', 'C': 0.1}

FINAL EVALUATION (after fitting the complete training set)
Training examples: 32561 | Test examples: 16281 | Features: 123
Selected parameters: {'kernel': 'linear', 'C': 0.1}
Mean CV accuracy: 84.7548%
Correct test predictions: 13841 / 16281
TEST ACCURACY: 85.0132%
Final solver fit_status: 0
Confusion matrix (rows=true, columns=predicted; labels=[-1,+1]):
  [11579, 856]
  [1584, 2262]

PASS: saved predictions match test labels, reported accuracy, and original log.
❖ (base) zhang@ZhangdeMacBook-Air question2 %

```

Figure 2: Original execution-log excerpts and verification of the final test result for the selected linear SVC with $C = 0.1$.

Question 3 (5 marks):

The optimization problem of linear soft-margin SVMs can be re-formulated as an instance of empirical structural risk minimization (refer to Page 37 on L5 notes). Show how to reformulate it. Hint: search reference about the hinge loss.

Answer:

1. Model and notation

Let the training set be

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N, \quad \mathbf{x}_i \in \mathbb{R}^d, \quad y_i \in \{-1, +1\}.$$

Define the real-valued decision score

$$f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{w}^\top \mathbf{x} + b, \quad \boldsymbol{\theta} = (\mathbf{w}, b),$$

where $\mathbf{w} \in \mathbb{R}^d$ is the weight vector and $b \in \mathbb{R}$ is the bias.

For training example i , define the functional margin

$$m_i = y_i f(\mathbf{x}_i; \boldsymbol{\theta}) = y_i (\mathbf{w}^\top \mathbf{x}_i + b).$$

The condition $m_i \geq 1$ means that the example is correctly classified and satisfies the margin requirement.

2. Constrained soft-margin SVM formulation

Following slide 36 of L5, the linear soft-margin SVM is

$$\begin{aligned}
 & \min_{\mathbf{w}, b, \xi} \quad \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^N \xi_i \\
 & \text{subject to} \quad y_i (\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad i = 1, \dots, N, \\
 & \quad \quad \quad \xi_i \geq 0, \quad i = 1, \dots, N,
 \end{aligned}$$

where $C > 0$ is the trade-off parameter and $\boldsymbol{\xi} = (\xi_1, \dots, \xi_N)^\top$ contains the slack variables.

The term $\frac{1}{2}\|\mathbf{w}\|_2^2$ controls model complexity, while $C \sum_{i=1}^N \xi_i$ penalizes violations of the margin requirement. The bias b is not regularized.

3. Eliminating the slack variables

For fixed $\mathbf{w}$ and b , the constraints on each slack variable can be rewritten as

$$\xi_i \geq 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b), \quad (9)$$

$$\xi_i \geq 0. \quad (10)$$

Equivalently,

$$\xi_i \geq \max \left\{ 0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b) \right\}.$$

Since $C > 0$, the objective is strictly increasing in each ξ_i when $\mathbf{w}$ and b are fixed. Therefore, the minimizing slack value is the smallest feasible one:

$$\xi_i^*(\mathbf{w}, b) = \max \left\{ 0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b) \right\}.$$

Define the hinge loss by

$$\ell_{\text{hinge}}(s, y) = \max\{0, 1 - ys\} = \begin{cases} 0, & ys \geq 1, \\ 1 - ys, & ys < 1. \end{cases}$$

Hence,

$$\xi_i^*(\mathbf{w}, b) = \ell_{\text{hinge}}(f(\mathbf{x}_i; \boldsymbol{\theta}), y_i).$$

For every $\mathbf{w}$ and b , choosing the slack variables as above satisfies the original constraints and minimizes their contribution to the objective. Substituting these values into the objective therefore gives the exactly equivalent unconstrained problem

$$\min_{\mathbf{w}, b} \frac{1}{2}\|\mathbf{w}\|_2^2 + C \sum_{i=1}^N \max \left\{ 0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b) \right\}.$$

4. Structural risk minimization formulation

Dividing the objective by the positive constant C does not change its minimizers. Thus, the problem can be written as

$$\min_{\mathbf{w}, b} \sum_{i=1}^N \ell_{\text{hinge}}(f(\mathbf{x}_i; \boldsymbol{\theta}), y_i) + \frac{1}{2C}\|\mathbf{w}\|_2^2.$$

Following slide 14 of L3, the structural risk minimization framework has the form

$$\min_{\boldsymbol{\theta}} \sum_{i=1}^N \ell(f(\mathbf{x}_i; \boldsymbol{\theta}), y_i) + \lambda \Omega(\boldsymbol{\theta}).$$

The soft-margin SVM matches this framework with

$$\ell(f(\mathbf{x}_i; \boldsymbol{\theta}), y_i) = \max \left\{ 0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b) \right\}, \quad (11)$$

$$\Omega(\boldsymbol{\theta}) = \|\mathbf{w}\|_2^2, \quad (12)$$

$$\lambda = \frac{1}{2C}. \quad (13)$$

Therefore, the linear soft-margin SVM minimizes the empirical hinge loss with squared L_2 regularization on $\mathbf{w}$. The hinge loss penalizes margin violations, including correctly classified examples with insufficient margin, while the regularizer controls model complexity. The bias b remains unregularized, consistently with the original formulation.

The data loss above is summed over the training examples. If empirical risk is instead expressed as an average, dividing the original objective by CN gives

$$\min_{\mathbf{w}, b} \frac{1}{N} \sum_{i=1}^N \ell_{\text{hinge}}(f(\mathbf{x}_i; \boldsymbol{\theta}), y_i) + \frac{1}{2CN} \|\mathbf{w}\|_2^2.$$

Under this convention, with $\Omega(\boldsymbol{\theta}) = \|\mathbf{w}\|_2^2$, the corresponding regularization coefficient is $\lambda = 1/(2CN)$.

Question 4 (5 marks):

Using the kernel trick introduced in L5 to extend the regularized linear regression model (L3) to solve nonlinear regression problems. Derive a closed-form solution (i.e., to derive a kernelized version of the closed-form solution on Page 50 of L3).

Answer:

1. Model and notation

Let the training set be

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N, \quad \mathbf{x}_i \in \mathbb{R}^d, \quad y_i \in \mathbb{R}.$$

Following the column-wise data convention in L3, define

$$X = [\mathbf{x}_1, \dots, \mathbf{x}_N] \in \mathbb{R}^{d \times N}, \quad \mathbf{y} = (y_1, \dots, y_N)^\top \in \mathbb{R}^N.$$

The linear regression model is

$$f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^\top \mathbf{x}.$$

We use the compact formulation in L3 without a separate unregularized intercept. A constant feature may be included in the feature vector, in which case its coefficient is also included in the stated L_2 penalty.

2. Regularized regression in feature space

Following slides 47–50 of L3, the regularized linear regression problem is

$$\min_{\mathbf{w}} E(\mathbf{w}) = \frac{1}{2} \|X^\top \mathbf{w} - \mathbf{y}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \quad \lambda > 0,$$

with the closed-form solution

$$\hat{\mathbf{w}} = (XX^\top + \lambda I_d)^{-1} X\mathbf{y}.$$

The data loss is summed over the training examples, consistently with L3.

To extend this model to nonlinear regression, introduce a nonlinear feature mapping φ . For matrix notation, first consider a finite feature dimension h :

$$\varphi: \mathbb{R}^d \rightarrow \mathbb{R}^h, \quad \Phi = [\varphi(\mathbf{x}_1), \dots, \varphi(\mathbf{x}_N)] \in \mathbb{R}^{h \times N}.$$

Let $\mathbf{w}_\varphi \in \mathbb{R}^h$ and define

$$f_\varphi(\mathbf{x}; \mathbf{w}_\varphi) = \mathbf{w}_\varphi^\top \varphi(\mathbf{x}).$$

This model is linear in the feature space but can be nonlinear in the original input space.

The regularized objective in the feature space is

$$E_\varphi(\mathbf{w}_\varphi) = \frac{1}{2} \sum_{i=1}^N \left(\mathbf{w}_\varphi^\top \varphi(\mathbf{x}_i) - y_i \right)^2 + \frac{\lambda}{2} \|\mathbf{w}_\varphi\|_2^2 \quad (14)$$

$$= \frac{1}{2} \|\Phi^\top \mathbf{w}_\varphi - \mathbf{y}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}_\varphi\|_2^2. \quad (15)$$

3. Expressing the solution using training examples

Differentiating with respect to $\mathbf{w}_\varphi$ gives

$$\nabla_{\mathbf{w}_\varphi} E_\varphi = \Phi(\Phi^\top \mathbf{w}_\varphi - \mathbf{y}) + \lambda \mathbf{w}_\varphi.$$

Setting the gradient to zero yields

$$(\Phi\Phi^\top + \lambda I_h) \hat{\mathbf{w}}_\varphi = \Phi \mathbf{y}.$$

Since $\lambda > 0$, the objective is strictly convex, so this stationary point is the unique minimizer. The corresponding feature-space solution is

$$\hat{\mathbf{w}}_\varphi = (\Phi\Phi^\top + \lambda I_h)^{-1} \Phi \mathbf{y}.$$

However, computing this expression directly requires explicit feature vectors and an $h \times h$ matrix.

To obtain a form suitable for the kernel trick, rearrange the stationarity condition as

$$\lambda \hat{\mathbf{w}}_\varphi = \Phi(\mathbf{y} - \Phi^\top \hat{\mathbf{w}}_\varphi).$$

Define

$$\hat{\boldsymbol{\alpha}} = \frac{1}{\lambda} (\mathbf{y} - \Phi^\top \hat{\mathbf{w}}_\varphi) \in \mathbb{R}^N.$$

Then the optimal weight vector is a linear combination of the mapped training examples:

$$\hat{\mathbf{w}}_\varphi = \Phi \hat{\boldsymbol{\alpha}} = \sum_{i=1}^N \hat{\alpha}_i \varphi(\mathbf{x}_i).$$

Substituting this representation into the definition of $\hat{\boldsymbol{\alpha}}$ gives

$$\lambda \hat{\boldsymbol{\alpha}} = \mathbf{y} - \Phi^\top \Phi \hat{\boldsymbol{\alpha}}, \quad (16)$$

$$(\Phi^\top \Phi + \lambda I_N) \hat{\boldsymbol{\alpha}} = \mathbf{y}. \quad (17)$$

4. Applying the kernel trick

Following slides 45–46 of L5, a kernel evaluates an inner product in the feature space:

$$k(\mathbf{x}, \mathbf{z}) = \varphi(\mathbf{x})^\top \varphi(\mathbf{z}).$$

Define the kernel matrix

$$K = \Phi^\top \Phi \in \mathbb{R}^{N \times N}, \quad K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j).$$

The coefficient equation therefore becomes

$$(K + \lambda I_N) \hat{\alpha} = \mathbf{y}.$$

For any nonzero $\mathbf{a} \in \mathbb{R}^N$,

$$\mathbf{a}^\top (K + \lambda I_N) \mathbf{a} = \|\Phi \mathbf{a}\|_2^2 + \lambda \|\mathbf{a}\|_2^2 > 0.$$

Thus, $K + \lambda I_N$ is positive definite and invertible, even if K itself is singular. The kernelized closed-form solution is

$$\hat{\alpha} = (K + \lambda I_N)^{-1} \mathbf{y}.$$

The associated feature-space weights satisfy

$$\hat{\mathbf{w}}_\varphi = \Phi (K + \lambda I_N)^{-1} \mathbf{y},$$

but neither these weights nor Φ need to be constructed explicitly.

5. Prediction and connection to linear regression

For a new input $\mathbf{x}$, define

$$\mathbf{k}(\mathbf{x}) = \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}) \\ \vdots \\ k(\mathbf{x}_N, \mathbf{x}) \end{bmatrix} \in \mathbb{R}^N.$$

The prediction is

$$\hat{f}(\mathbf{x}) = \hat{\mathbf{w}}_\varphi^\top \varphi(\mathbf{x}) \tag{18}$$

$$= \sum_{i=1}^N \hat{\alpha}_i \varphi(\mathbf{x}_i)^\top \varphi(\mathbf{x}) \tag{19}$$

$$= \sum_{i=1}^N \hat{\alpha}_i k(\mathbf{x}_i, \mathbf{x}). \tag{20}$$

Hence,

$$\hat{f}(\mathbf{x}) = \mathbf{k}(\mathbf{x})^\top (K + \lambda I_N)^{-1} \mathbf{y}.$$

The fitted values on the training set are

$$\hat{\mathbf{y}} = K (K + \lambda I_N)^{-1} \mathbf{y}.$$

Both training and prediction depend only on kernel evaluations. The final kernel formulation also applies to kernels with infinite-dimensional feature maps, without explicitly constructing those features.

For the linear kernel $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$, we have $K = X^\top X$ and $\mathbf{k}(\mathbf{x}) = X^\top \mathbf{x}$. Using

$$(X X^\top + \lambda I_d) X = X (X^\top X + \lambda I_N),$$

the recovered weight vector is

$$\hat{\mathbf{w}} = X (X^\top X + \lambda I_N)^{-1} \mathbf{y} = (X X^\top + \lambda I_d)^{-1} X \mathbf{y},$$

which is exactly the closed-form solution on slide 50 of L3. Replacing the linear kernel with a nonlinear kernel therefore extends regularized linear regression to nonlinear regression while preserving a closed-form training solution.